0543. Diameter of Binary Tree

Easy | Category: Trees | LeetCode: https://leetcode.com/problems/diameter-of-b

Problem

- Return the length (in edges) of the longest path between any two nodes. The path need not pass through the root.

Intuition

- The longest path "bends" at some node: it goes down the left subtree and down the right subtree.
- Path through node = height(left) + height(right). Compute heights bottom-up and track the max of that sum.

Brute Force

- For each node compute heights separately: O(n^2).

Optimal Approach

- Height(null) = 0.
- At each node: l = height(left), r = height(right); best = max(best, l + r); return 1 + max(l, r).

Worked Example

```
[1,2,3,4,5].

height(4)=1 height(5)=1 -> at 2: path 1+1=2, height 2
height(3)=1             -> at 1: path 2+1=3, height 3
diameter = 3 (4-2-1-3)
```

Java Solution

```java
public class DiameterOfBinaryTree {
    private int best;

    public int diameterOfBinaryTree(TreeNode root) {
        best = 0;
        height(root);
        return best;
    }

    private int height(TreeNode n) {
        if (n == null) return 0;
        int l = height(n.left), r = height(n.right);
        best = Math.max(best, l + r);
        return 1 + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n). Space: O(h).

Pitfalls

- Return height but record diameter: two different quantities from one DFS.